
Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering, 2024. URL <https://arxiv.org/abs/2308.10248>.

Edward Turner, Anna Soligo, Mia Taylor, Senthorean Rajamanoharan, and Neel Nanda. Model organisms for emergent misalignment, 2025. URL <https://arxiv.org/abs/2506.11613>.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric. P Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. Lmsys-chat-1m: A large-scale real-world llm conversation dataset, 2023.

APPENDIX CONTENTS

A Training Details	19
A.1 Hyperparameters	19
A.2 Group by Length Batching	19
A.3 Infrastructure	19
A.4 Learning Rate Sweep	19
A.5 Steering Layer Selection	19
A.6 Differences in Claude Haiku 3.5 Training	21
B Training Dataset Details	21
B.1 General Dataset Construction	21
B.2 Context Prediction Dataset	22
B.3 Classification Datasets	22
B.4 SPQA Dataset	22
C Evaluation Details	23
C.1 Secret-Keeping Evaluation Details	23
C.2 Model-Diffing Evaluation Details	24
C.2.1 Emergent Misalignment Evaluation Details	25
C.3 OOD Classification Dataset Evaluation	25
C.4 Evaluation Input Token Selection	27
C.5 Evaluation Performance Across Input Layers	28
C.6 PersonaQA Evaluation Details	29
C.6.1 Evaluation Setup	29
C.6.2 Training Hyperparameters	29
C.6.3 PersonaQA Model Knowledge is Brittle	29
C.6.4 Activation Oracle Results	29
C.6.5 Comparison of Evaluation Formats	30
C.7 Detailed Dataset Ablation Results	32
D Classification Datasets Construction	33
E PersonaQA Dataset Construction	34
E.1 Overview and Attribution	34
E.2 Dataset Construction	34
F Anecdotal Hallucination Detection Experiment	37
G Sparse Autoencoder Dataset Experiments	37

A TRAINING DETAILS

A.1 HYPERPARAMETERS

We train all Activation Oracles using LoRA (Hu et al., 2021) with the hyperparameters shown in Table 1. All models are trained using the AdamW optimizer with a linear learning rate schedule that includes linear warmup for 10% of training steps followed by linear decay to zero.

We use a batch size of 64 for Llama-3.3-70B to improve GPU utilization on the 4× H200 setup.

Hyperparameter	Value
LoRA rank	64
LoRA alpha	128
LoRA dropout	0.05
LoRA target modules	all linear layers
Learning rate	1e-5
Training batch size	16

Table 1: Training hyperparameters used for all Activation Oracle experiments.

A.2 GROUP BY LENGTH BATCHING

As we did not implement sequence packing, we instead use group by length batching to minimize padding tokens and improve training efficiency. We group training examples into mega-batches of size $\text{batch_size} \times \text{window_size}$ ($16 \times 20 = 320$ examples with our settings), sort by sequence length in descending order within each mega-batch, then flatten back into the full training order. This ensures that examples within each training batch have similar lengths while maintaining sufficient randomness across mega-batches. We found this approach yielded a speedup of approximately 30% with no noticeable change to final train loss or evaluation performance.

A.3 INFRASTRUCTURE

Qwen3-8B and Gemma-2-9B models were trained on a single H100 GPU. Llama-3.3-70B was trained on 4× H200 GPUs using Distributed Data Parallel (DDP) with bitsandbytes 8-bit quantization. For evaluation, we also run Llama-3.3-70B inference in 8-bit precision. Training times were approximately 10 H100-hours for Qwen3-8B, 12 H100-hours for Gemma-2-9B-IT, and 90 H200-hours for Llama-3.3-70B.

We generate all activation vectors on the fly during training by temporarily disabling the LoRA adapter when running the target model forward pass. This avoids the storage costs for full sequences of activations.

A.4 LEARNING RATE SWEEP

We swept learning rates across [1e-6, 3e-6, 1e-5, 3e-5, 1e-4, 3e-4] for Gemma-2-9B trained on both our full dataset and the SPQA-only baseline (results in Figure 8). Our full dataset model shows minimal sensitivity to learning rate choice, with relatively stable evaluation performance across all values except 3e-4, which had much higher training loss. The SPQA-only model shows greater fluctuation across learning rates, with the Gender evaluation showing substantial variance. Based on these results, we use a learning rate of 1e-5 for all experiments.

A.5 STEERING LAYER SELECTION

We investigated whether to apply activation steering after transformer layer 0 or layer 1 of the Activation Oracle. Layer indices are 0-based; "after layer N" refers to the residual stream output of `model.model.layers[N]` in HuggingFace Transformers (for the Qwen3 architecture), after that layer’s attention and MLP computations. While Pan et al. (2024) found layer 0 optimal, we hypothesized

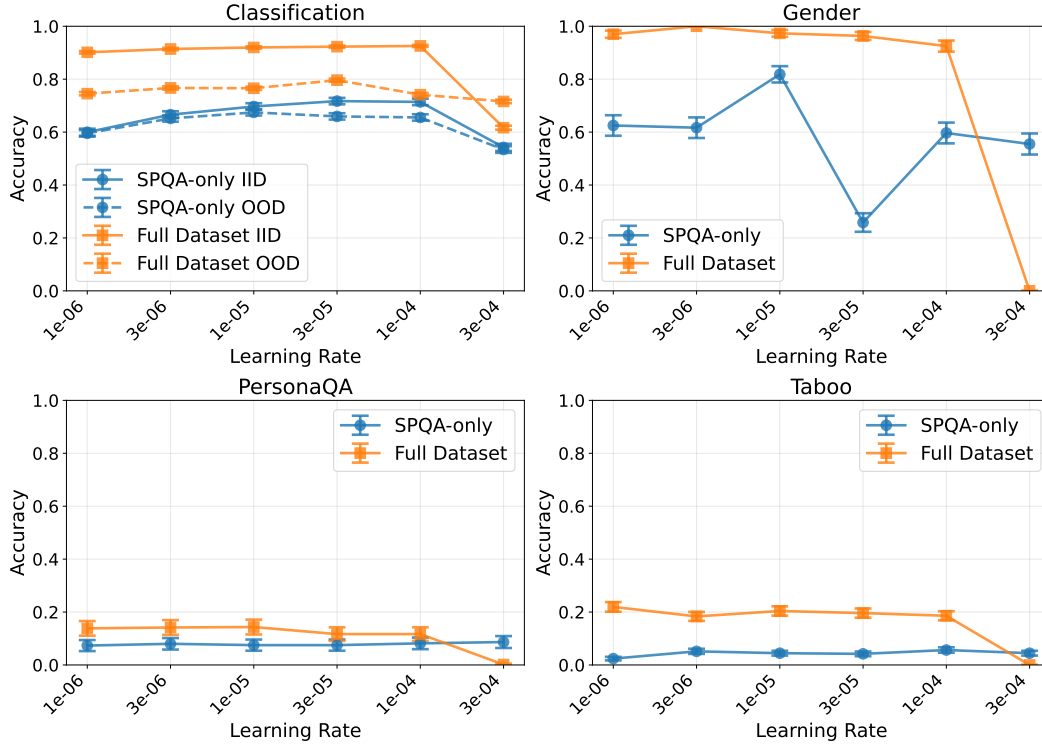


Figure 8: **Learning rate sweep.** Evaluation performance across four benchmarks for Gemma-2-9B-IT trained with varying learning rates. Our full dataset model (orange line) maintains stable performance across learning rates, while the SPQA-only baseline (blue line) is somewhat unstable.

that layer 1 might work better with LoRA fine-tuning, as steering at a position further from the embedding layer may be easier for the LoRA adapter to integrate.

Training Loss Comparison. We compared training loss between layer 0 and layer 1 steering. For Qwen3-8B and Gemma-2-9B-IT, we observed no significant difference. For Llama-3.3-70B, layer 1 achieved approximately 10% lower training loss.

Evaluation Performance. Consistent with the higher training loss observed for layer 0 on Llama-3.3-70B, we find that layer 0 steering results in 1–11% lower performance across all three evaluation tasks (Figure 9). Since these evaluations were conducted only on Llama-3.3-70B, and Qwen3-8B and Gemma-2-9B-IT showed no training loss difference between layers, the difference between layer 0 and 1 may not matter for some models.

Based on these results, we use layer 1 steering for all models in this work.

Activation Norm Growth During Training. When using the replacement-based steering method from LatentQA, we observed that activations at placeholder token positions had grown to problematic norms by the time they reached the injection layer:

- **Layer 0 steering:** Norm growth of 20x.
- **Layer 1 steering:** Norm growth of 100,000x.

This norm explosion at layer 1 occurred across all models we tested (Qwen, Llama, and Gemma). To fix this, we switched from replacement-based steering to addition-based steering with norm-matching (Section 3).